

Vibe-IC — 全部步驟：Phase → Stage → Step (v2.2.0 • 繁體中文)

一份清單：整條流程的每一步，以 **Phase → Stage → Step** 階層組織，單一連續編號 1 → 33（跨 Stage 1-4，自 Stage 1 的 Spec-to-RTL 起算）。Phase 1 的文件生成步驟以字母 **D1-D5** 標示（前置，不計入 1→33）。兩條並行支線 —— Analog (A1-A9) 與 Mixed-signal (M1-M4) —— 同時進行。真實來源 = runner；更細的程式層標記自動產生於 `FLOW_STEPS_GENERATED.md`。

Phase → Stage 對照

- **Phase 1** — 規格與文件 → 兩條匯流入口：**Agent 路徑** (PM Agent • IC Expert Agent) • **doc-gen 路徑** (D1-D5)
- **Phase 2** — RTL → 合成 → Stage 1 (RTL+驗證) • Stage 2 (合成+DFT)
- **Phase 3** — 實體 → Tapeout → Stage 3 (實體+簽核) • Stage 4 (輸出+Tapeout)
- **並行** — Analog A1-A9 • Mixed-signal M1-M4

Phase 1 — 規格與文件

Phase 1 有**兩條匯流的入口**，都產出同一批 L1-L23 層文件。刻意**不設 Phase 1a/1b 子編號** —— 依 RFC v2.0 § 8.2，入口差異只是一個輸入子目錄、而非 phase 切分，因為兩條路徑最終都產出 L1-L13：

- `phase1/input_prompt/` —— 自由文字 / 自然語言 → **Agent 路徑** (PM Agent + IC Expert Agent)。
- `phase1/input_doc/` —— 既有文件 / 結構化 YAML → **決定性 doc-gen 路徑** (D1-D5)。

兩者匯流到餵給 Phase 2 的 L1-L23 文件。

Agent 路徑 —— `phase1/input_prompt/` • 2 個 Agent Skills

當輸入是自由文字時，由兩個 agent 驅動 Phase 1。PM Agent 面對使用者；IC Expert Agent 在其後方運作，**從不直接與使用者對話**。

#	Agent Skill	角色	面對使用者？
1	PM Agent (<code>agents/pm-agent.md</code>)	自然語言前門：NL-ingest → 缺口對話（一次問一題、用使用者的語言）→ 確認 fact graph → 交棒。把使用者的產品語言轉成 L1-L9 事實。	✔ 是
2	IC Expert Agent (<code>agents/ic-expert-agent.md</code>)	PM 後方的矽智財審查者：審查每一層的技术完整性、填入自動決定的預設值（附 <code>auto_decided + reasoning</code> 軌跡）、跨層一致性檢查 (L5↔L4 腳位、L6↔L5 訊號、L9↔L5+L6+L8)、套用設計保守原則。 這是 plugin 的 IC 專業知識倉庫 —— 它的逐層審查清單就是領域知識累積之處（每一筆	✗ 否 —— 僅透過 PM Agent

#	Agent Skill	角色	面對使用者？
		benchmark-enhancement-capture 的 Bucket-B 學習都落在這裡）。	

交棒鏈：使用者自由文字 → **PM Agent**（NL-ingest + 缺口對話）→ fact graph → **IC Expert Agent**（審查 / 填值 / 跨層一致性）→ 定稿 L 系列文件。**IC Expert Agent** 是 **plugin** 隨時間累積矽智財知識之處；PM Agent 則全程讓非專家使用者維持在白話層次。交棒後的事實再走下方同一套 doc-gen 萃取器。

決定性 doc-gen 路徑 —— phase1/input_doc/ • D1-D5（前置 • 不計入 1→33）

#	步驟	工具 / 方式
D1	匯入與文字萃取（prompt 或既有文件 → input_doc/）	phase1_doc_one_shot_runner
D2	產生 L1-L13 核心設計層文件	決定性萃取器
D3	產生 L14-L23 協定 / 時序 / skeleton 文件	overlay 萃取器
D4	協定類別合成 dispatch（81 類）	is_<proto> + <proto>_synth
D5	Coverage / parity 報告	phase1 parity 報告

Phase 2 — RTL → 合成

Stage 1 — RTL 產生與驗證

#	步驟	工具 / 方式
1	Spec-to-RTL（由 L-docs 撰寫 RTL）	spec-to-rtl skill
2	Lint	eda_lint + hygiene gates
3	CDC / RDC check	cdc-check
4	Simulation	testbench-gen + eda_simulate
5	Formal verification	formal-verify + assertion-gen
6	FPGA early prototype	eda_fpga_compile / eda_fpga_program → .sof

Stage 2 — 合成與 DFT

#	步驟	工具 / 方式
7	Constraint setup	constraint-gen → *.sdc
8	SDC validation	SDC lint
9	Synthesis (Yosys)	eda_synth + synth-doctor
10	Pre-layout STA	eda_sta_mcorner (SS/TT/FF)
11	DFT insertion (scan + ATPG)	dft-insert + atpg + eda_dft
12	Post-DFT optimization	resynth / buffering

#	步驟	工具 / 方式
13	Equivalence check (LEC)	equivalence-check + Yosys equiv

Phase 3 — 實體設計 → Tapeout

Stage 3 — 實體設計與簽核

#	步驟	工具 / 方式
14	Floorplan + PDN	eda_pnr (init)
15	Clock planning	clock-planning skill
16	Placement (global + detailed)	eda_pnr
17	CTS	eda_pnr enable_cts=true
18	Post-CTS hold fixing	repair_timing -hold
19	Routing (global + detailed)	eda_pnr enable_detailed_route=true
20	寄生萃取 → SPEF	OpenRCX extract_parasitics
21	Post-route STA (MMMC)	eda_sta_mcorner
22	IR drop	OpenROAD PSM analyze_power_grid
23	EM check	PSM -enable_em
24	Antenna check	OpenROAD check_antennas
25	Signal integrity (crosstalk)	SPEF coupling-cap 篩查
26	Post-layout gate-level sim (+SDF)	eda_simulate
27	Physical verification (DRC / LVS / ERC + PERC-equivalent)	KLayout DRC + LVS 簽核鏈 + Magic ERC + perc_equivalent
28	ECO repair loop	eco-plan

Stage 4 — 輸出與 Tapeout

#	步驟	工具 / 方式
29	Power analysis	pre + post layout
30	Metal fill (density fill)	OpenROAD filler_placement
31	Tapeout checklist	tapeout-checklist + signoff_audit
32	GDSII output	eda_gds + def2gds
33	FPGA final sign-off	recompile + on-board test + attestation

並行支線

Analog A1-A9 (analog_one_shot_runner.py，與 Stage 1-3 並行)

#	步驟	工具 / 方式
A1	spec_extract	→ A1_spec.json

#	步驟	工具 / 方式
A2	topology_select	analog-topology-select → A2_topology.json
A3	netlist_gen	→ <block>.sp
A4	corner_sweep	ams-sim → A4_corners.json
A5	layout (Magic)	→ A5_layout.json
A6	逐區塊實體驗證 (DRC + LVS)	analog_a6_block_pv_check
A7	post-layout resim	→ A7_postsim.json
A8	hardmacro_gen	→ {.lef, .lib, .gds, .v} (餵回 Stage 3)
A9	hw_verify (HIL) / co-sim	→ A9_hw_verify.json

Mixed-signal M1-M4 (mixed-signal-cosim skill，無專屬 runner)

#	步驟	工具 / 方式
M1	top merge	mixed_signal_m1_top_merge_check.py
M2	co-sim setup	mixed-signal-cosim skill
M3	co-sim run	mixed-signal-cosim skill
M4	integration verify	mixed-signal-cosim skill

總計

Phase	Stages	Steps
Phase 1 — 規格與文件	兩條入口 (Agent • doc-gen)	D1-D5 + PM Agent • IC Expert Agent
Phase 2 — RTL → 合成	Stage 1 • Stage 2	1-13
Phase 3 — 實體 → Tapeout	Stage 3 • Stage 4	14-33
並行	Analog • Mixed-signal	A1-A9 • M1-M4

33 個循序步驟 (Stage 1 : 1-6 • Stage 2 : 7-13 • Stage 3 : 14-28 • Stage 4 : 29-33) ，外加 **Phase 1** (兩條入口：Agent 路徑 —— PM Agent • IC Expert Agent —— 與 doc-gen 路徑 D1-D5) 與兩條並行支線。

預檢：P0 (mcp_server_health_check、eda_doctor) 。編排器 vibe_ic_one_shot_runner.py 跑 Phase 1 → Phase 2 → Analog → Phase 3 。

摘要之外的細節：runner 即時程式層標記見 FLOW_STEPS_GENERATED.md ；LVS 簽核鍵與 PERC-equivalent 覆蓋見 PERC_SIGNOFF_MEMO.md 。英文正本：ALL_STEPS_v2.2.0.md 。